

Week03 | 作业 | 采集最小链路 v1 与《采集链路 Runbook》v1

本页结构

把 Week03 的最小采集闭环整理成正式交付物	1
这次作业真正要交付什么	2
先把实验页和作业页的边界分开	2
先看一张“正式交付清单”图	3
建议投入时间	3
完成标准	3
最终必交工件	3
Week03 在项目里的位置	4
你必须围绕 repo 现有基线交付	5
契约	5
Manifest	5
Ingest 入口	5
验证入口	5
任务 1: 收口 Week03 的 ingest baseline 设计	5
你需要特别说清的一件事	6
任务 2: 完成 batch ingest 主链路设计说明	6
建议你用这个表来组织内容	6
任务 3: 完成 incremental / checkpoint 策略说明	6
这部分最容易得分的做法	6
任务 4: 完成资产化设计与恢复策略说明	7
asset_flow_plan_v1.md 应至少回答	7
replay_backfill_strategy_v1.md 应至少回答	7
任务 5: 完成 Runbook v1	7
建议至少覆盖这 3 类故障场景	8
任务 6: 写一页交付总结	8
推荐验证命令	8
推荐目录结构	9
评分参考	9
最低验收清单	9
一条推荐 workflow	10
选做加分项	10
这次作业真正让你获得什么	10
下一步会发生什么	10

把 Week03 的最小采集闭环整理成正式交付物

这份作业不是让你重复实验页的每一步，而是要求你把 Week03 真正交付成一套：

可解释、可验证、可恢复、可交接 的最小采集链路。

这次作业真正要交付什么

Week03 到这里已经把一条最小 ingest 基线的关键部件讲完了：

- 课时 1：为什么 ingest 可靠性决定下游可复现性
- 课时 2：为什么 batch ingest 必须具备幂等、可重跑和完整性校验
- 课时 3：为什么 incremental / CDC 不能轻易承诺 exactly-once
- 课时 4：为什么要从任务流转向资产流
- 课时 5：为什么 recovery thinking 和 runbook 必须前置
- 实验：你已经围绕当前 repo baseline 跑过一次最小闭环

这次作业不是把实验机械重做一遍。它真正要求你交付的是一套 **Week03 输入采集与恢复基线包**：

让别人接手你的仓库，也能知道 ingest 从哪里开始、如何验证、出问题后怎么判断、怎么恢复。

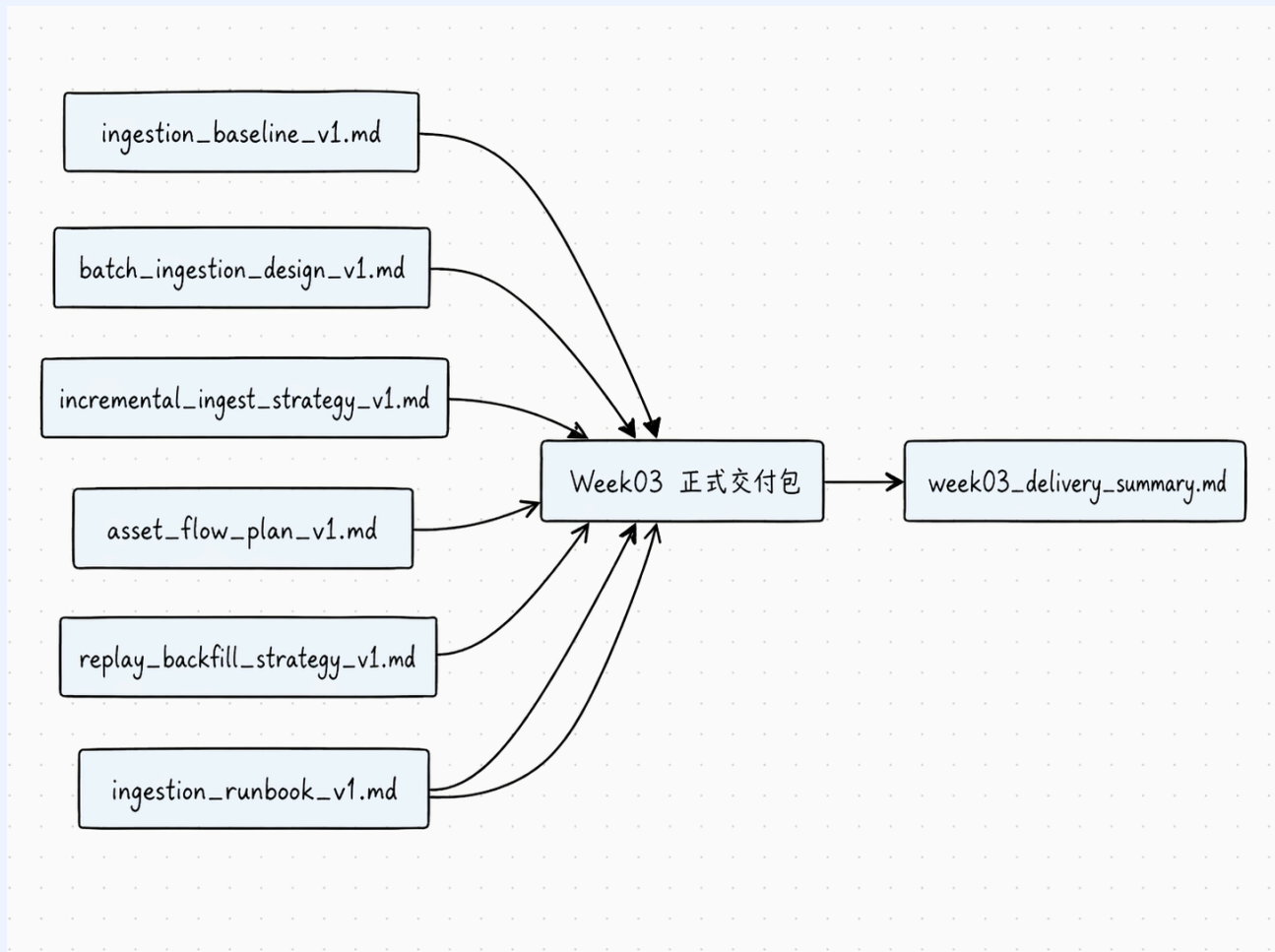
先把实验页和作业页的边界分开

页面	最核心的问题
实验页	这条最小闭环到底能不能被验证，验证的是哪些不变量
作业页	这条最小闭环怎样被整理成正式交付物，供别人接手、复核和继续迭代

所以这次作业不应该写成“我把实验又跑了一遍”，而应该写成：

- 哪条 baseline 被正式定义了
- 哪些恢复边界被明确写进文档
- 哪些工程判断可以被团队继续复用

先看一张“正式交付清单”图



你可以把这张图理解成：

- 实验页更像“证据收集”
- 作业页更像“正式归档”
- 最后交付的不只是几份 markdown，而是一套能被别人继续接手的 Week03 基线包

建议投入时间

4–6 小时

完成标准

！不要把“命令跑过了”当成作业完成

这次作业真正的通过标准是：

1. 你能交付一套可读、可执行、可追溯的 Week03 ingest baseline。
2. 你的交付物和当前 OmniSupport Copilot repo 结构保持一致，而不是另起一套平行世界。
3. 你能解释：为什么这条链路是 `batch baseline`，为什么 `incremental` / CDC 只推进到边界与策略层。
4. 你能给出最小的恢复判断与 Runbook，而不是只说“重跑一下看看”。

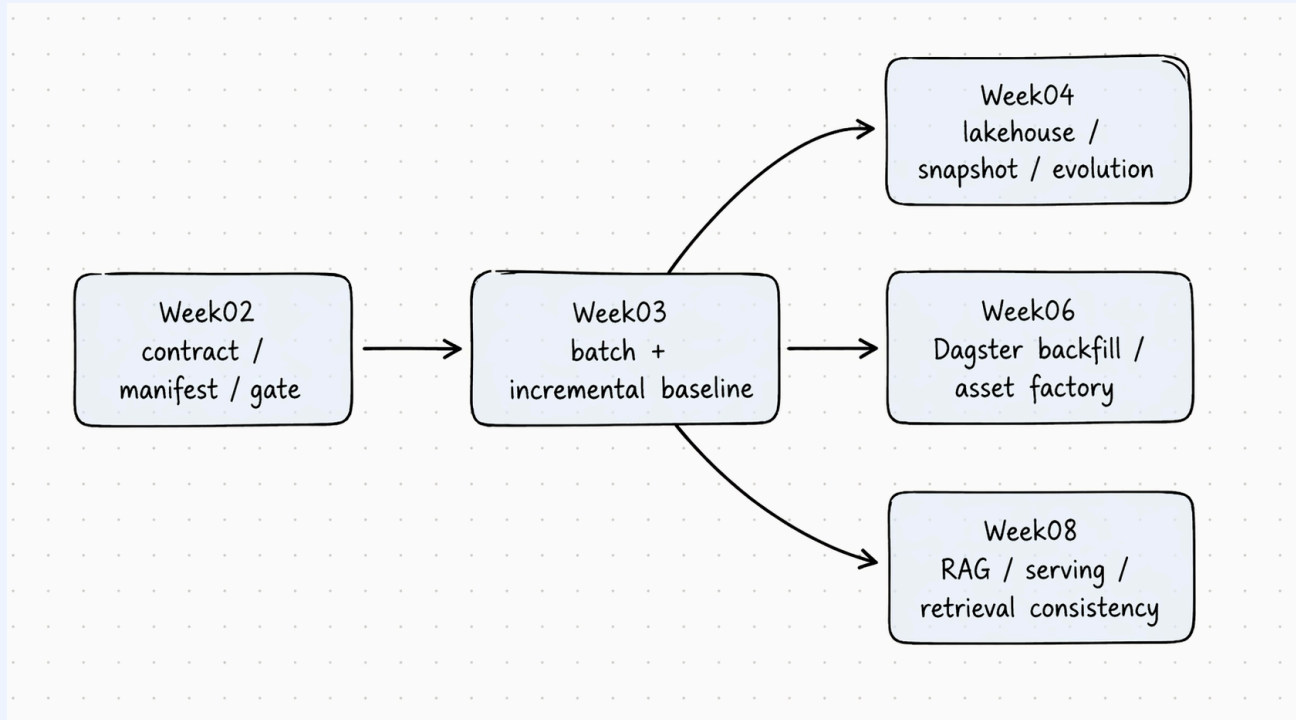
最终必交工件

请至少提交下面这些文件：

类型	必交文件	作用
设计蓝图	docs/blueprints/week03/ingestion_baseline_v1.md	说明 Week03 最小 ingest 基线的目标、路径、边界
Batch 设计	docs/blueprints/week03/batch_ingestion_design_v1.md	说明双源 batch ingest 的主链路
Incremental 设计	docs/blueprints/week03/incremental_ingest_strategy_v1.md	说明 cursor / checkpoint / late arrival 的处理原则
资产化设计	docs/blueprints/week03/asset_flow_plan_v1.md	说明 ingest 如何接入 Dagster asset graph
恢复策略	docs/blueprints/week03/replay_backfill_strategy_v1.md	说明 replay / rerun / backfill 的适用边界
Runbook	runbooks/ingestion_runbook_v1.md	交付给团队的采集链路应急与恢复手册
结果记录	reports/week03/week03_delivery_summary.md	用自然语言总结本周交付与关键判断

如果你已经在课时和实验中生成了部分模板，可以继续完善后提交。这次作业不要求你重造文件，而是要求你把本周内容收口成一套**正式交付物**。

Week03 在项目里的位置



这张图的重点是：

- Week02 定义输入准入边界
- Week03 让输入真正进入系统
- Week03 的结果会继续被 Week04、Week06、Week08 消费

所以你这次提交的不是课堂笔记，而是 后续周次继续复用的基础设施说明。

你必须围绕 repo 现有基线交付

这次作业必须继续贴着当前项目仓库的真实对象：

契约

- `contracts/data/ticket_contract.json`
- `contracts/data/doc_asset_contract.json`
- `contracts/data/audio_asset_contract.json`
- `contracts/data/video_asset_contract.json`

Manifest

- `data/seed_manifests/manifest_tickets_synthetic_v1.json`
- `data/seed_manifests/manifest_workspace_helpcenter_v1.json`
- `data/seed_manifests/manifest_edge_gateway_pdf_v1.json`
- `data/seed_manifests/source_manifest_schema.json`

Ingest 入口

- `pipelines/ingestion/seed_loader.py`
- `pipelines/ingestion/ticket_ingest.py`
- `pipelines/ingestion/doc_ingest.py`
- `pipelines/ingestion/assets.py`

验证入口

- `tests/contract/test_json_schemas.py`

i 这次作业不要求你重写 repo 基线

你不需要为了作业再发明另一套 contract / manifest / gate 系统。你要做的是：

- 读懂当前 repo baseline
- 验证它
- 补齐说明
- 明确边界
- 把恢复与运行规则沉淀为团队可用文档

任务 1：收口 Week03 的 ingest baseline 设计

完成：

`docs/blueprints/week03/ingestion_baseline_v1.md`

这份文档至少要回答 5 个问题：

1. Week03 的最小 ingest baseline 到底覆盖哪些 source?
2. 为什么学生本地主线先走 batch / incremental cursor，而不强制完整 Kafka / Debezium 栈?
3. 当前 manifest / contract / seed loader / tests 各自负责什么?
4. 这条最小链路和 Week04 / Week06 / Week08 怎么衔接?
5. 这条基线目前的能力边界在哪里?

你需要特别说清的一件事

本周不是“不讲 CDC / Stream”，而是：

在当前课程主线里，优先把 batch baseline、incremental boundary、replay / backfill thinking 做稳。完整 CDC / Stream 能力，可以作为后续周或更大规模实现继续扩展。

任务 2：完成 batch ingest 主链路设计说明

完成：

[docs/blueprints/week03/batch_ingestion_design_v1.md](#)

这份文档至少写清楚：

- 你这周选择的双源是什么，建议是 `ticket + document`
- 每条链路的输入是什么
- 每条链路最小输出是什么
- 哪些步骤是 `dry-run`
- 哪些结果可以作为完整性检查依据

建议你用这个表来组织内容

链路	输入	最小校验	预期输出	失败后怎么处理
<code>ticket_ingest</code>	synthetic ticket JSONL	contract / required fields / business rule	re-raw_ticket_event / ticket_fact base-line	rerun / replay
<code>doc_ingest</code>	manifest source dir	+ contract / meta-data / file presence	raw_doc_asset / knowledge_doc baseline	rerun / backfill

任务 3：完成 incremental / checkpoint 策略说明

完成：

[docs/blueprints/week03/incremental_ingest_strategy_v1.md](#)

这里不要装作 repo 已经 fully 实现 `exactly-once`。你真正要交的是：

- 哪个字段可以作为 `cursor`
- 哪些 `source` 适合做 `incremental`
- 为什么 `replay / backfill` 要分开
- 如何处理 `late-arriving data`
- 为什么当前阶段更合理的目标是：
 - `at-least-once ingest`
 - `idempotent write`
 - `dedupe`
 - `replayable recovery`

这部分最容易得分的做法

不是写很多术语，而是把下面这张表写清楚：

概念	你在本项目里怎么理解
<code>cursor</code>	哪个字段驱动增量边界
<code>checkpoint</code>	成功处理到哪里
<code>watermark</code>	当前批次承认的时间边界
<code>late arrival</code>	到得比窗口晚的数据
<code>replay</code>	重新消费同一批或同一来源
<code>restore</code>	把系统或数据拉回某个已知可用状态
<code>backfill</code>	补历史空洞 / 旧分区

任务 4：完成资产化设计与恢复策略说明

完成这两个文件：

```
docs/blueprints/week03/asset_flow_plan_v1.md
docs/blueprints/week03/replay_backfill_strategy_v1.md
```

`asset_flow_plan_v1.md` 应至少回答

- 当前 Dagster ingest asset graph 有哪些资产
- `seed_manifests`、`raw_doc_assets`、`raw_ticket_events` 分别在表达什么
- 为什么 manifest 不是 asset 本身，但会影响 asset materialization
- 为什么 Week03 先让学生理解资产流，而不是让他们一上来写复杂 scheduler

`replay_backfill_strategy_v1.md` 应至少回答

- `retry` / `rerun` / `replay` / `restore` / `backfill` 的区别
- 什么情况适合 `replay`，什么情况适合 `backfill`
- 什么情况必须先 `restore`，再决定后续 `replay` 或 `backfill`
- 为什么“出问题就全量重跑”不是成熟答案
- 现阶段 repo 基线下，最合理的恢复动作是什么

任务 5：完成 Runbook v1

完成：

```
runbooks/ingestion_runbook_v1.md
```

这份 runbook 不要求很长，但必须可执行。建议至少包含以下章节：

1. 适用范围
2. 依赖服务
3. 最小健康检查
4. 标准执行命令
5. 常见失败现象
6. 恢复动作选择表
7. 何时升级为人工介入
8. 运行后如何记录

建议至少覆盖这 3 类故障场景

场景	你应该写什么
<code>seed_loader</code> 通过, 但某条 source dry-run 失败	如何判断是 manifest 问题还是源内容问题
<code>ticket_ingest</code> dry-run 出现 schema / business rule 问题	应该 rerun、replay 还是先修 source
<code>doc_ingest</code> 缺文件 / metadata 不完整	应该补文件、补 manifest, 还是整体 backfill

任务 6: 写一页交付总结

完成:

```
reports/week03/week03_delivery_summary.md
```

这份总结建议至少有 5 段:

1. 这次 Week03 你真正交付了什么
2. 双源 ingest 的最小链路长什么样
3. 你如何理解 incremental / replay / backfill 的边界
4. 当前 repo baseline 最值得保留的工程优势是什么
5. 如果进入 Week04, 你最希望补的一个能力是什么

这份总结不是额外负担, 而是把工程判断表达清楚的能力证明。

推荐验证命令

在提交前, 至少跑过这些命令:

```
docker compose --env-file infra/env/.env.local -f infra/docker-compose.yml up -d --build
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    pytest tests/contract/ -v
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python -m pipelines.ingestion.seed_loader --manifest-dir data/seed_manifests
```

如果你已经按实验页往下做, 也建议保留下面这些命令的运行记录:

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python -m pipelines.ingestion.ticket_ingest --dry-run --input ...
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python -m pipelines.ingestion.doc_ingest --dry-run --manifest ...
```


推荐目录结构

如果你还没整理 Week03 的作业文件，建议至少收口成这样：

```
omnissupport-copilot/  
  contracts/  
    data/  
      ticket_contract.json  
      doc_asset_contract.json  
      audio_asset_contract.json  
      video_asset_contract.json  
    data/  
      seed_manifests/  
        manifest_tickets_synthetic_v1.json  
        manifest_workspace_helpcenter_v1.json  
        manifest_edge_gateway_pdf_v1.json  
        source_manifest_schema.json  
      docs/  
        blueprints/  
          week03/  
            ingestion_baseline_v1.md  
            batch_ingestion_design_v1.md  
            incremental_ingest_strategy_v1.md  
            asset_flow_plan_v1.md  
            replay_backfill_strategy_v1.md  
        runbooks/  
          ingestion_runbook_v1.md  
      reports/  
        week03/  
          week03_delivery_summary.md
```

评分参考

维度	分值	看什么
repo 对齐度	25	是否真正围绕当前 OmniSupport Copilot repo 基线展开
ingest 设计清晰度	20	是否说清了双源 batch ingest 的结构与边界
incremental / recovery 判断	20	是否正确区分 cursor / checkpoint / replay / back-fill
runbook 可执行性	20	是否真的写成了可执行步骤，而不是概念摘要
工程表达能力	15	总结、图表、文档组织是否清晰稳定

最低验收清单

在提交前，请逐项自查：

- ☐ 我已经完成 Week03 的 5 份 blueprint 文档
- ☐ 我已经完成 `runbooks/ingestion_runbook_v1.md`
- ☐ 我已经完成 `reports/week03/week03_delivery_summary.md`
- ☐ 我跑过至少 3 条 Week03 基线命令
- ☐ 我能解释为什么当前课程主线先不强制完整 Kafka / Debezium 栈
- ☐ 我能解释 `retry / rerun / replay / restore / backfill` 的区别
- ☐ 我能说明这些交付物如何进入 Week04 / Week06

一条推荐工作流

如果你想按最稳的顺序完成，建议这样走：

1. 先复查实验页的结果
2. 再写 `ingestion_baseline_v1.md`
3. 再写 `batch_ingestion_design_v1.md`
4. 再写 `incremental_ingest_strategy_v1.md`
5. 再写 `asset_flow_plan_v1.md`
6. 再写 `replay_backfill_strategy_v1.md`
7. 然后完成 `ingestion_runbook_v1.md`
8. 最后写 `week03_delivery_summary.md`

这样做的好处是：你的 runbook 和 summary 会建立在已经梳理过的工程判断上，而不是写成空泛总结。

选做加分项

如果你想把 Week03 作业做得更像真实项目，可以继续做下面任意一项：

1. 给 `ingestion_runbook_v1.md` 增加故障分级 例如 P1 / P2 / P3，对应不同恢复策略。
2. 在 `incremental_ingest_strategy_v1.md` 中补一张 state 结构示意图 让 checkpoint / cursor / watermark 更容易被团队理解。
3. 在 `asset_flow_plan_v1.md` 中补一张 asset dependency 图 明确 `seed_manifests` -> `raw_doc_assets` / `raw_ticket_events` 的关系。
4. 在 `week03_delivery_summary.md` 中补一节“Week04 风险前瞻”说明 snapshot / schema evolution / compaction 为什么会紧接着成为 Week04 主题。

这次作业真正让你获得什么

如果你认真完成这次作业，你得到的不是几份 md 文件，而是一套更成熟的工程心智：

- 知道如何把 contract 推进成 ingest baseline
- 知道为什么批处理不是落后方案，而是可靠基线
- 知道增量和 CDC 的边界应该怎么讲、怎么落
- 知道为什么 recovery thinking 必须在系统早期就带进来
- 知道 runbook 不是上线后补的，而是 Week03 就应该出现的工程资产

下一步会发生什么

Week04 会继续沿着这条线走下去，但焦点会从“数据怎么进来”转成：

- 这些数据在 Lakehouse 里怎么稳定存放
- 如何用快照、schema evolution、时间旅行支撑复现

- 为什么 snapshot / rollback / compaction 会成为 AI 数据工程的核心基础能力
也就是说，Week03 的作业写得越扎实，Week04 越不会变成“突然多了一堆新名词”。

[返回 Week03 总览](#) | [回看实验](#)